



SAP Application Development (S/4 HANA)

Dynamic Programs

Motivation

This chapter will show you the structure of more complex programs with integrated Dynpros. You will create a new program which selects and shows flight details from the SAP flight example. Therefore you will use the Dynpro technology.

Prerequisites

Before starting the exercises, you should have fundamental programming skills and be familiar with ABAP development in Eclipse. Furthermore, it is recommended that you have completed the chapters *Development of a first application*, *Basic concepts of ABAP programming*, and *Database access with ABAP programs*.

Content

As the major part of the SAP transactions are still developed using ABAP and Dynpros, this chapter is one of the most important ones to understand the program logic of SAP. For this, first a connection to the SAP system is established. Afterwards, you create your first Dynpro program and an initial screen for your application. This is followed by a second screen, which provides a more complex functionality in terms of data manipulation through an input screen.

Lecture notes

Students can go on with their account from the previous chapters mentioned in the prerequisites. If you skipped these chapters, your students may need to enter a developer key when creating their first program. Please see the lecture notes for further assistance on this topic. Furthermore, students should always use an individual three-digit number to mark their repository objects. In this exercise, these are displayed as ### and have to be replaced by the students' individual numbers. The institutional code is represented as \$\$ and has to be replaced as well.

- **Product:** All
- **Level:** Beginner
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

Task 1: Start Eclipse and connect to the SAP system

Short description: Use Eclipse to connect to the SAP system with your username and password

Start Eclipse, and select your workspace. In the Project Explorer, double-click on your ABAP project. Please refer to the Introduction chapter if you have no current ABAP project in the Project Explorer and want to create a new one. After double-clicking, a popup appears, with which you can connect to the SAP system by entering the password for your user.

Login for Project DEV_100_dev-000_en

Login for Project DEV_100_dev-000_en

Enter login data for system DEV: ABAP development

Client: * 100

User: * DEV-000

Password: * [masked]

Language: * EN

OK Cancel

Login

You should now see the two folders **Favorite Packages** and **System Library** below your ABAP project. If you have already created an ABAP package in one of the previous exercises, make sure that your package is shown in the **Favorite Packages** folder, and go on to Task 2.

If not, please create a new Package. For this purpose, right click on your ABAP project and choose **New->ABAP Repository Object**. Select **Package** as new repository object and click on **Next**.

New ABAP Repository Object

ABAP Repository Object

Create an ABAP Repository Object

Project: DEV_100_dev-000_en Browse

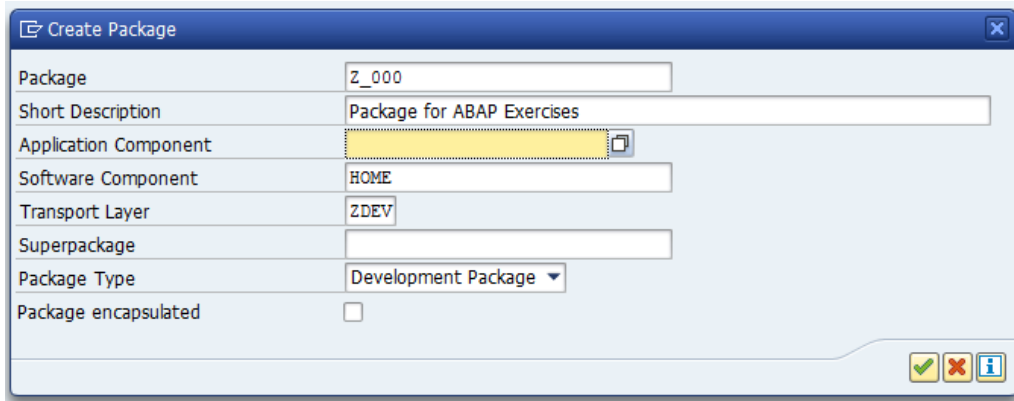
Package

- Package
 - Package
- Package Interfaces
 - Package Interface

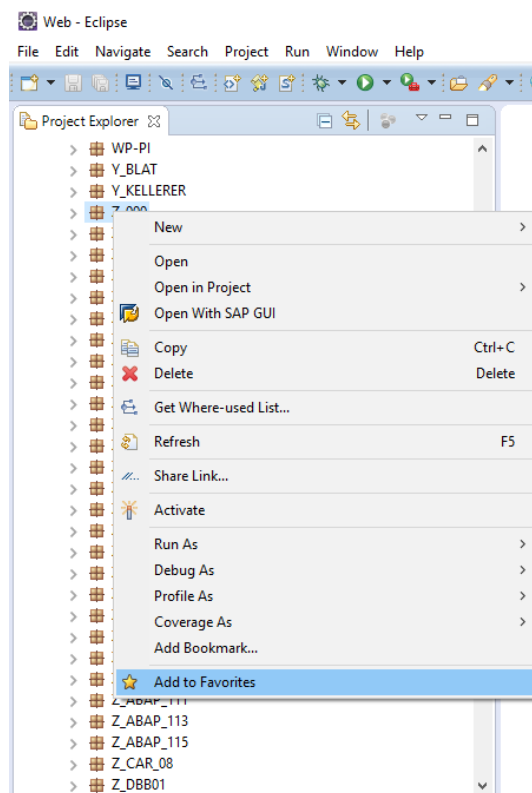
< Back Next > Finish Cancel

Create new Package

Enter the name **Z_\$\$_###** for your package and click on **Finish**. Now the integrated SAP GUI will open. Enter the Short Description for your package and click on . Select a transport request for your package (or create a new one by clicking on  and determining a description), and click enter.



Now your ABAP package will be shown in the **System Library** folder (You may need to refresh the folder). Look it up, and add it to your favorite packages.



**Add to
Favorite
Packages**

Task 2: Create your first program with Dynpros

Short description: Create a program which uses two Dynpros to display flight details from the SAP flight example

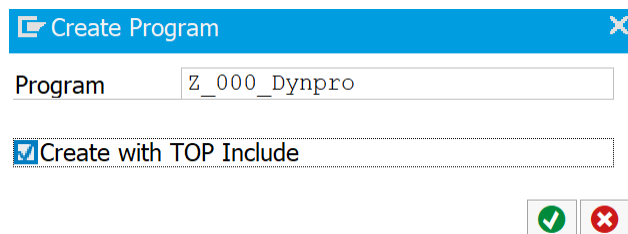
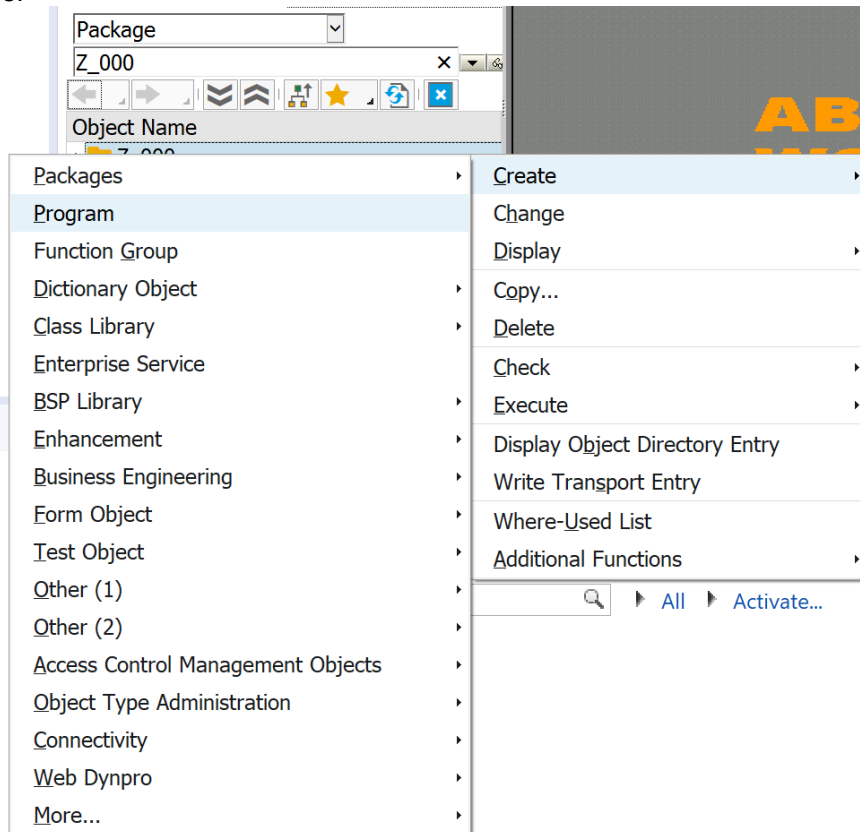
Please start the integrated SAP GUI in Eclipse by clicking on , choosing your ABAP Project, and selecting **OK**.

Open the ABAP Development Workbench by calling transaction code **SE80**. Search for your package **Z_\$\$_###** in the Explorer of the integrated SAP GUI (located on the left).

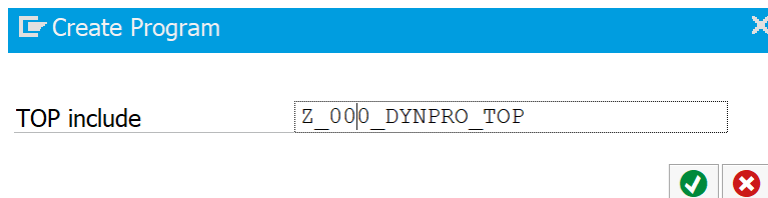
**Open ABAP
Development
Workbench**

In the context menu of your package in the Explorer of the integrated SAP GUI (Right-click), create a new program called **Z_\$\$_###_DYNPRO**. Please use **Create with TOP Include**.

TOP INCL



Modify the name of your top include to **Z_\$\$_###_DYNPRO_TOP**.



Hint:

Please note that the SAP system automatically proposed a name for your top include which did not start with the obligatory Z to mark it as a customer top include. Later on in the development you will often face such situations when the SAP system proposes a name which is not in the customer namespace. Pay attention to this!

Hint

In the next popup you see all program attributes. You have to adjust these attributes. Select the status **Test Program** and the application **Cross-Application**. Continue and assign the new program to your existing package **Z_\$\$_###** and your transport request.

Do not wonder why the SAP system asks you for a transport request twice: first for the new program and the second time for the top include.

ABAP: Program Attributes Z_001_DYNPRO Change

Title: Include Z_000_DYNPRO

Original language: EN English

Created: DEV-000 07.09.2017

Last Changed:

Status:

Attributes

Type: Module Pool

Status: Test Program

Application: Cross-Application

Authorization Group:

☐ Editor lock ☒ Fixed point arithmetic

☒ Unicode Checks Active

Save Undo Redo

**Assign
attributes**

In the Project Explorer of Eclipse, open your new top include in the folder **Source Library->Includes** (located in your package Z_\$\$_###) by double-clicking on it. The source code of the top includes opens in the Editor Area. In this step we want to define all global variables. Such variables are available in the entire program and every screen has access to them. These variables are the table **SPFLI** (which represents an instance of the flight table), the system variable **ok_code** and the work area **wa_flight**. Please define them in the top include:

```
PROGRAM Z_000_DYNPRO.

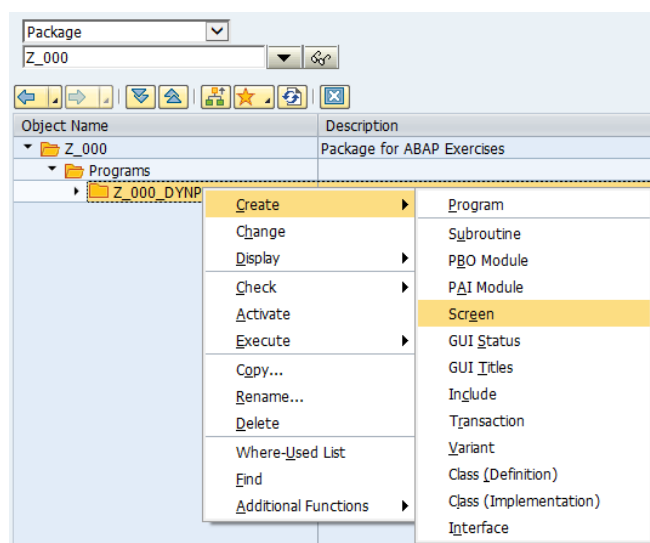
Tables spfli.

DATA: ok_code LIKE sy-ucomm, wa_flight TYPE spfli.
```

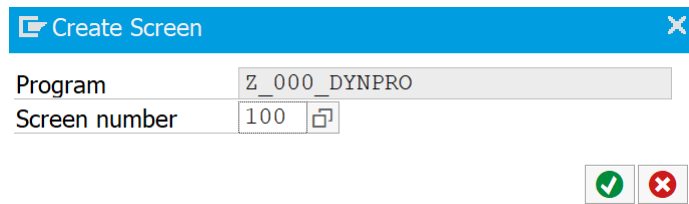
**Maintain
variables**

Save your source code by either clicking on the **Save** or the **Save All** button .

The next step contains the creation of the first Dynpro. Go back to your program in the integrated SAP GUI. Right click on your program name in the navigation tree and create a new Dynpro (screen) with the screen number 100.



**Create
Dynpro**



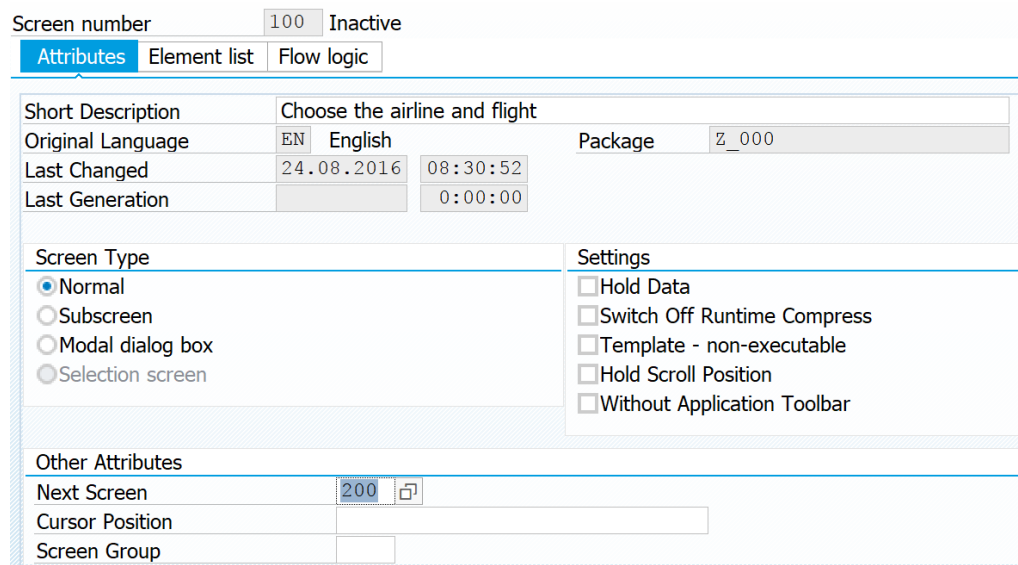
Create Screen

Program: Z_000_DYNPRO

Screen number: 100

Buttons: [OK] [Cancel]

Now you have to specify the attributes of your new Dynpro. Maintain the short text and fill in the Dynpro 200 as the **Next Screen**. This will let the Dynpro jump to Dynpro 200 after Dynpro 100 was processed. Of course, you have not created the Dynpro 200 yet, but you will do so later on in the exercise.



Screen number: 100 Inactive

Attributes | Element list | Flow logic

Short Description: Choose the airline and flight

Original Language: EN English Package: Z_000

Last Changed: 24.08.2016 08:30:52

Last Generation: 0:00:00

Screen Type:

- ☒ Normal
- ☐ Subscreen
- ☐ Modal dialog box
- ☐ Selection screen

Settings:

- ☐ Hold Data
- ☐ Switch Off Runtime Compress
- ☐ Template - non-executable
- ☐ Hold Scroll Position
- ☐ Without Application Toolbar

Other Attributes:

Next Screen: 200

Cursor Position:

Screen Group:

Maintain attributes

Save your screen by clicking on the Save  button in the integrated SAP GUI. Change from change to display mode by clicking on . Afterwards, in the Eclipse Tool Bar, click on **Activate inactive ABAP development objects** . In the popup, select your development objects and click **Activate**.

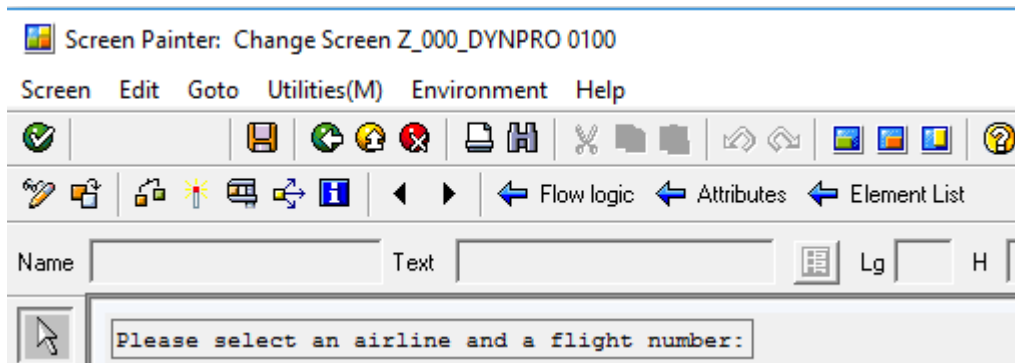
Task 3: Create the first screen for your Dynpro

Short description: Create a screen which allows you to select an airline and a flight number

Your next step is the creation of the layout of your first Dynpro. Therefore, double-click on your screen 100 in the Explorer of the integrated SAP GUI, and call the Graphical Screen Painter by pressing the button  **Layout** in the toolbar. As long as the Graphical Screen Painter is active, you cannot access the integrated SAP GUI. You can return to the SAP GUI by pressing the **Back** button in the Graphical Screen Painter.

Call Graphical Screen Painter

If necessary, please change to **Change** mode by clicking on . Select the text field button  and draw a text field on your new screen. Use **TXT_INSTRUCTION** as the name of the text field and type in the text: '**Please select an airline and a flight number:**'. Your screen should now look similar to this:



The user of your new program should choose an airline and a flight number in this screen. This information is stored in the table **SPFLI** from the SAP flight example. To create two input fields with an entry help, you reference to the table **SPFLI** from the data dictionary. This can be done very easily by pressing the button **Dictionary/Program fields window** in the toolbar. Type in the name **SPFLI** and press the **Get from Dictionary** button. You will notice that the SAP system reads the information from the dictionary.

Call dictionary

Table/Field Name	Description	I/O Field	None	Short	Medium	Long	Header	Text	Copy as
SPFLI MANDT		☒ CLNT 3	☐	☐	☐	☐	☐	Client	☐
SPFLI CARRID		☒ CHAR 3	☐	☐	☐	☐	☐	Airline	☐
SPFLI CONNID		☒ NUMC 4	☐	☐	☐	☐	☐	Flight Number	☐
SPFLI COUNTRYFR		☒ CHAR 3	☐	☐	☐	☐	☐	Country	☐
SPFLI CITYFROM		☒ CHAR 20	☐	☐	☐	☐	☐	Depart.city	☐
SPFLI AIRPFROM		☒ CHAR 3	☐	☐	☐	☐	☐	Dep. airport	☐
SPFLI COUNTRYTO		☒ CHAR 3	☐	☐	☐	☐	☐	Country	☐
SPFLI CITYTO		☒ CHAR 20	☐	☐	☐	☐	☐	Arrival city	☐
SPFLI AIRPTO		☒ CHAR 3	☐	☐	☐	☐	☐	Dest. airport	☐
SPFLI FLTIME		☒ INT4 6	☐	☐	☐	☐	☐	Flight time	☐
SPFLI DEPTIME		☒ TIMS 8	☐	☐	☐	☐	☐	Departure	☐
SPFLI ARRTIME		☒ TIMS 8	☐	☐	☐	☐	☐	Arrival Time	☐
SPFLI DISTANCE		☒ QUAN 11	☐	☐	☐	☐	☐	Distance	☐
SPFLI DISTID		☒ UNIT 3	☐	☐	☐	☐	☐	Distance in	☐
SPFLI FLTYPE		☒ CHAR 1	☐	☐	☐	☐	☐	Charter	☐
SPFLI PERIOD		☒ INT1 3	☐	☐	☐	☐	☐	n day(s) later	☐

As your application should provide the user with two input fields for the airline and the flight number, please select the **CARRID** and **CONNID** row from the popup and press enter. This changes your mouse cursor and lets you drop two new input fields on your screen. So far your screen should look similar to this:

Select CARRID und CONNID

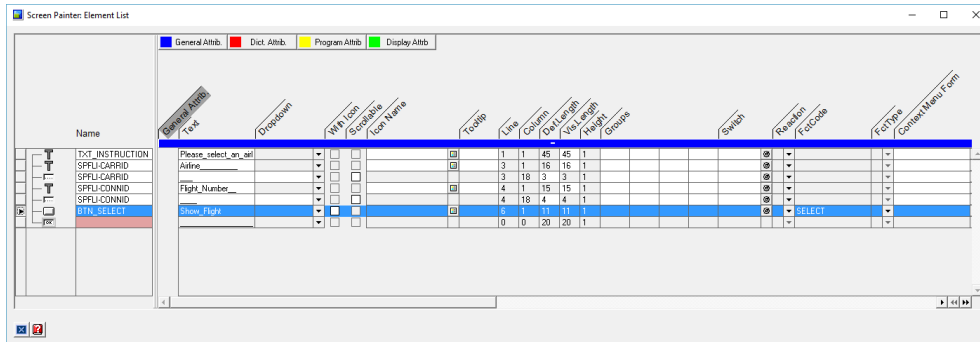
In the last step we need a button to go on to our second Dynpro. So please add a button to your screen and name it **BTN_SELECT** and choose a button text on your own. After you entered the information you will notice that the button is still colored red. This means that the SAP system is missing some information.

Hint:

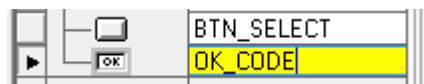
If an element in the Graphical Screen Painter is colored red, this indicates that the Graphical Screen Painter is missing information. By double clicking on the red element you can get further information.

Hint

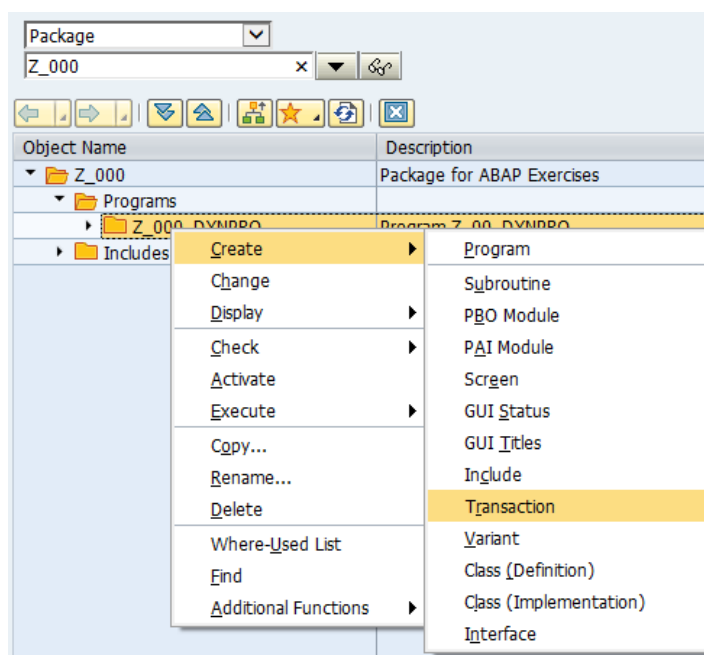
As your new button is still colored red you have to add some needed information. The needed information is the function code. Please double click on your button. This opens the attributes view and you discover the red input field where you have to insert the function code **SELECT** and press enter. Later in your program, pressing this button should automatically assign the value **SELECT** to your variable **ok_code**. The variable is available everywhere in your program as this variable was declared in the top include. But until yet the link between the function code and the variable **ok_code** is still missing. To establish the link click on the **Element List** button  in the toolbar. This will open a new window with all the elements from your current screen.

**Element list**

Once again you see a red field indicating that there is still something missing. Next to the red field there is a little OK-symbol which represents the variable **ok_code**. Please type in **OK_CODE** which will enable the link to the global variable **ok_code**.

OK_CODE

Please save your new Dynpro by clicking  and return to the Development Workbench. For that, close the Screen Painter, or click on Back. To simplify the life of your user, we want to create a transaction code for our new program. This is done by right clicking on your program in the navigation tree and choosing the menu point **Create->Transaction**:

**Create transaction**

The SAP system comes up with a new popup where you can specify the transaction code as well as a description.

Create Transaction
✕

Transaction code Z_000_DYNPRO

Transaction attributes

Short text Call Program Z_000_DYNPRO

Start object

☒ Program and screen (dialog transaction)
☐ Program and selection screen (report transaction)
☐ Method of a class (OO transaction)
☐ Transaction with variant (variant transaction)
☐ Transaction with parameters (parameter transaction)

✓
✕

Please note that the transaction has nothing to do with the called program. The link between the transaction code and the program is done later when specifying the transaction attributes. This means: you may call a program with a completely different name than the transaction code. The little example of the Development Workbench with transaction code SE80 is a very good example of this independency.

In the attributes of the transaction code (which will appear after you clicked enter), you can now specify the called program. Please type in the name of your Dynpro program **Z_\$\$_###_DYNPRO** and enter the screen number **100**. Activate the options **SAPGUI for HTML** and **SAPGUI for Windows**. Please save your transaction by clicking  (you have to select your package and transport request) and activate all of your developments by clicking on .

Transaction code Z_000_DYNPRO

Package Z_000

Transaction text

Call Program Z_000_DYNPRO

Program Z_000_DYNPRO

Screen number 100

Authorization Object
Values

☒ Maintenance of standard transaction variant allowed

Classification

Transaction classification

☒ Professional User Transaction
☐ Easy Web Transaction Service
☐ Pervasive enabled

GUI support

☒ SAPGUI for HTML
☐ SAPGUI for Java
☒ SAPGUI for Windows

Specify attributes

To test your new transaction code and your new program, create a new SAP GUI window by clicking on  in the integrated SAP GUI. Call your program using the transaction code `Z_$$_###_DYNPRO`. You will see your text field and two input fields. The first one is equipped with an entry help, whereas the second one is not. When hitting the button, you will get an ABAP dump (Automatically generated error object that is shown after the program terminated because of an error, which isn't handled by a suitable exception class). This absolutely makes sense as you defined the screen 200 as the next screen of screen 100 and as you have not created the screen 200 yet your program runs into a problem.

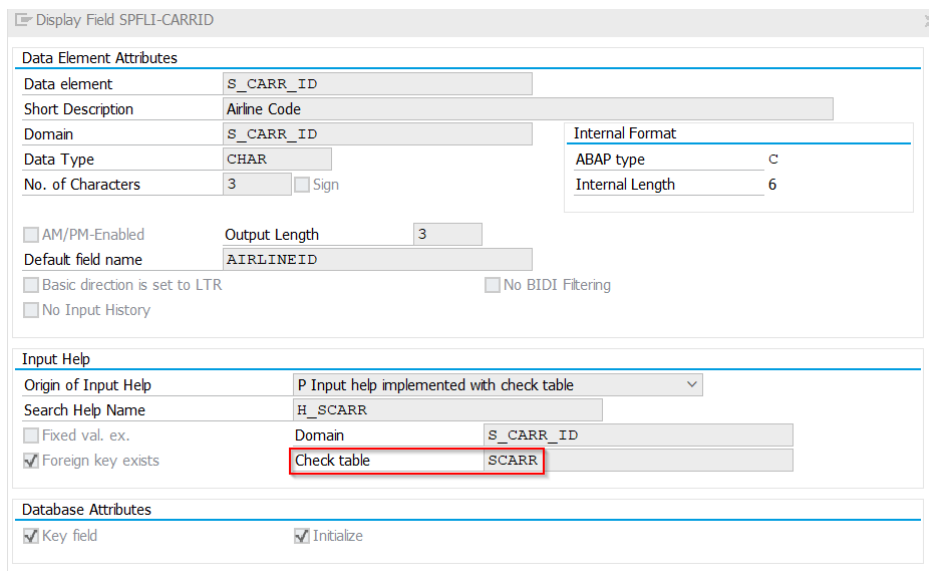
Test and get an ABAP dump

Go back to your Dynpro program in the integrated SAP GUI and switch to the **Flow Logic** tab of your screen 100. You find it in the **Screens** folder. When you open the flow logic tab, you see the flow logic of your first Dynpro, separated into a process before output module and a process after input module. We will use the commented modules later to implement what should happen before the screen is displayed and after the user has triggered a function key. Save the program by clicking , and activate it :

```
PROCESS BEFORE OUTPUT.
  * MODULE STATUS_0100.
  *
PROCESS AFTER INPUT.
  * MODULE USER_COMMAND_0100.
```

Check

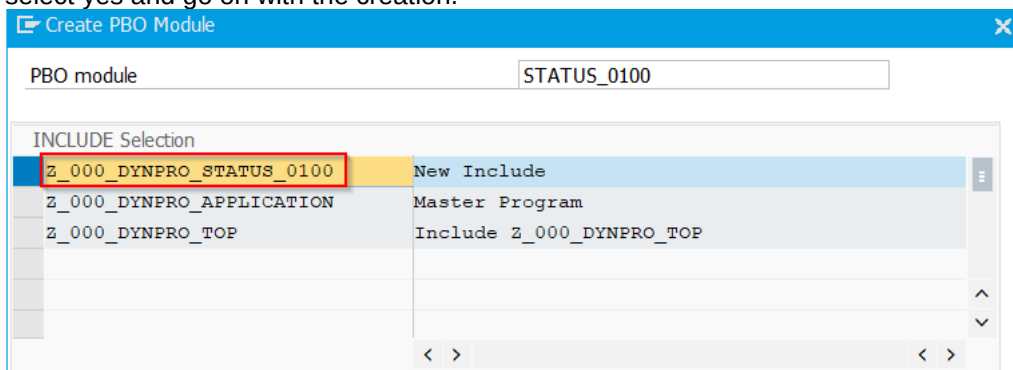
Please start your Dynpro program again and test the behavior of your program. You will notice that only the first field is equipped with an entry help. The reason for that is simple. Open transaction **SE11** (ABAP Dictionary) and display the table **SPFLI**. As you know the first field was defined in the screenpainter based on the table field **CARRID**. Double click on the **CARRID** field to display its properties. In the **section "Input Help"** you will notice that there is a value assigned to **Check table**. The reason is, that **CARRID** is a foreign key that references to the table **SCARR**. Due to this relationship the ABAP runtime environment is able to propose you with values based on this check table.



Data Element Attributes	
Data element	S_CARR_ID
Short Description	Airline Code
Domain	S_CARR_ID
Data Type	CHAR
No. of Characters	3
AM/PM-Enabled	☐
Output Length	3
Default field name	AIRLINEID
Basic direction is set to LTR	☐
No Input History	☐
No BIDI Filtering	☐
Input Help	
Origin of Input Help	P Input help implemented with check table
Search Help Name	H_SCARR
Fixed val. ex.	☐
Foreign key exists	☒
Domain	S_CARR_ID
Check table	SCARR
Database Attributes	
Key field	☒
Initialize	☒

Close the popup window and display the properties of field **CONNID**. As you can see **CONNID** is not a foreign key and no check table is assigned to it. Hence this field doesn't offer any default input help. Later in this exercise we will implement our own input help for this field. For now, please go back to the flow logic of your screen 0100. You will now focus on the status bar and the title of your new program. Therefore, please uncomment the `MODULE STATUS_0100` line in the flow logic by removing the little * and double click

on **STATUS_0100**. You will be asked if you want to create the new module. Please select yes and go on with the creation.



The system automatically proposes a name for your new status module. The name of your new PBO module is fine, but the name for the PBO module-include is not. Please change the name and remove the last three characters if necessary (cf. screenshot above). After you hit **Enter** the system comes up with a short message telling you that an additional include-statement is inserted into your program. This is completely fine as you create a new include for the status bar and this include must be included in your Dynpro program. Double click on the new module STATUS_0100 in the PBO Modules to navigate forward to the source code of the module.

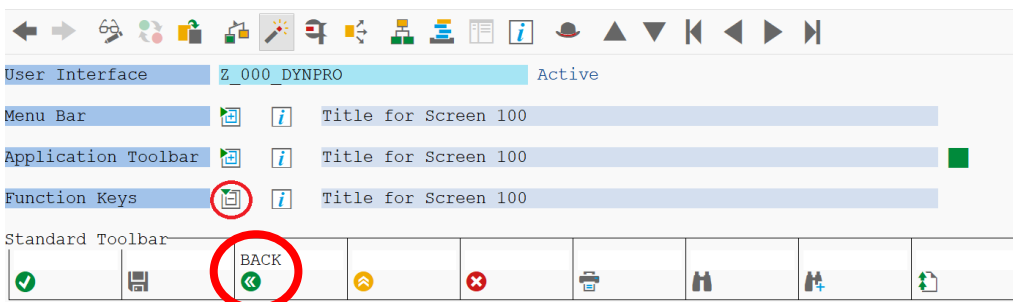
**Rename
include**

```
MODULE status_0100 OUTPUT.
  SET PF-STATUS '100'.
  SET TITLEBAR '100'.
ENDMODULE.
```

Uncomment

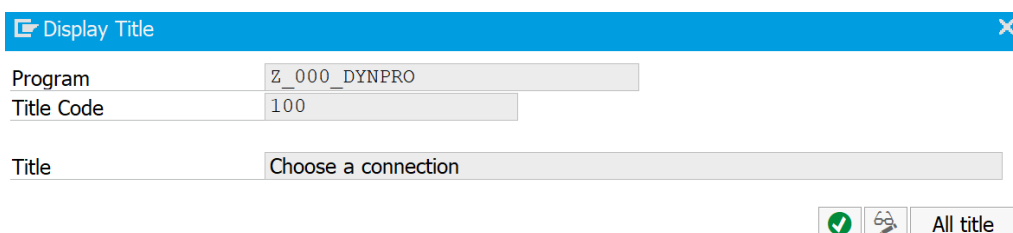
Uncomment both SET lines and replace the placeholder 'xxx' with the number '100'. Now you set up the status bar and then the title bar for your screen. This is done by double clicking on the '100' after **PF-STATUS**. You are asked if you want to create the status. Please answer yes and maintain a short text in the next pop-up (e.g. "Title for Screen 100"). After that pop-up the SAP system comes up with the following screen:

PF-STATUS



Please click on  next to Function Keys to get more details. Here you can create, edit and delete the status bar for you screen. You may change the code which is parsed to the program for every button in the status bar. Those buttons, where no code is assigned are unavailable later on. Please maintain the code for the **Back** button by adding **BACK**. **Save** your new status bar and return to the program code of your module by clicking Back. Double click on the **100** after **TITLEBAR**.

TITLEBAR



Maintain the title of your new title bar. Save, and activate **all** of your new objects, then test your new program in an additional GUI window. It should look similar to this:

Choose a connection

Please select an airline and a flight number:

Airline 

Flight Number

Next Screen

Now as you have created the layout you should create the flow logic as well. So please go to your screen **100** and switch to the tab **Flow Logic**. Uncomment the line **MODULE USER_COMMAND_0100** and double click on it. You are asked by the system if you want to create the object. Please answer yes and maintain the name of the include: delete the last three entries of the name and add **Z_\$\$_###_** at the beginning if it's not already proposed by the system.

**USER_COMM
AND_0100**

Create PAI Module

PAI module USER_COMMAND_0100

INCLUDE Selection

Z_000_DYNPRO_USER_COMMAND_0100	Include
Z_000_DYNPRO_APPLICATION	Master Program
Z_000_DYNPRO_STATUS_0100	Include Z_000_DYNPRO_STATUS_0100
Z_000_DYNPRO_TOP	Include Z_000_DYNPRO_TOP



Assign the new module to your existing package. In the next step you have to add the flow logic to your new module. Finally, this step can be done without the integrated SAP GUI, and you can make use of the code completion of Eclipse. In the Project Explorer, go the **Includes** folder of your program and open **Z_\$\$_###_USER_COMMAND_100** by double-clicking on it. The flow logic has to handle the **ok_code** in your program. Depending on the value of the **ok_code** the program either should leave the execution (**BACK**) or read some data from table SPFLI (**SELECT**). Please insert a case instruction. Notice how variable names are proposed for the fields of the **spfli** table.

**CASE
instruction**

```
MODULE user_command_0100 INPUT.
CASE ok_code.
  WHEN 'BACK'.
    LEAVE PROGRAM.
  WHEN 'SELECT'.
    SELECT SINGLE * FROM spfli INTO wa_flight WHERE carrid = spfli-carrid AND
      connid = spfli-connid.
ENDCASE.
CLEAR ok_code.
ENDMODULE.
```

Please save and activate all of your objects.

Hint:

Please note that the CASE instruction is case sensitive. This means that it makes a difference if you use 'back' or 'BACK' in your case instruction! This is a mistake, that happens very often.

Hint**Task 4: Create the second screen for your Dynpro**

Short description: Create the second screen in your new program to display the flight details

You have created the first screen in your program for the user to choose a connection, but you need a second screen to display the connection details. This is done in this task.

Please create a new screen in your program with the number **200** using the integrated SAP GUI. Please refer to the second task if you have problems. Use screen **100** as the **Next screen**.

Screen 200

Screen number Active

Attributes Element list Flow logic

Short Description Screen to display connection details

Original Language EN English Package Z_210

Last Changed 17.08.2017 14:43:52

Last Generation 17.08.2017 14:43:57

Screen Type

☒ Normal

☐ Subscreen

☐ Modal dialog box

☐ Selection screen

Settings

☐ Hold Data

☐ Switch Off Runtime Compress

☐ Template - non-executable

☐ Hold Scroll Position

☐ Without Application Toolbar

Other Attributes

Next Screen 

Cursor Position

Screen Group

Lines/Columns Occupied 0 0

Mainten. 27 120

Context Menu FORM ON CTMENU

 Properties

Please switch to the layout of your new screen  **Layout**. In the Graphical Screen Painter, you are going to add some input fields to your screen as you want to display the connection details. But instead of creating the text fields step-by-step you refer to the data dictionary again . Please enter the table name **SPFLI**, hit **ENTER** and select every field from the table by using this little button: . Then go on.

Edit Layout

Screen Painter: Dict./Program Fields

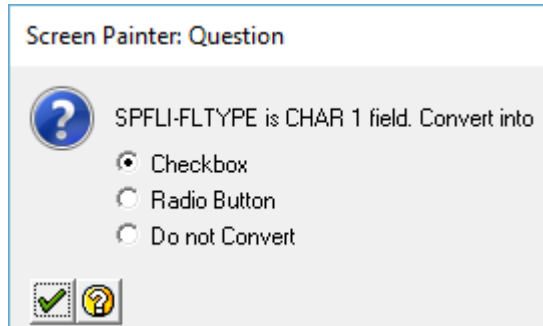
Table/Field Name  

Switch Reaction RFC Destination

Table/Field Name	Description	I/O Field	None	Short	Medium	Long	Header	Copy as
SPFLI	MANDT	3	☒	☐	☐	☐	☐	Client
SPFLI	CARRID	3	☒	☐	☐	☐	☐	Airline
SPFLI	CONNID	4	☒	☐	☐	☐	☐	Flight Number
SPFLI	COUNTRYFR	3	☒	☐	☐	☐	☐	Country
SPFLI	CITYFROM	20	☒	☐	☐	☐	☐	Depart city
SPFLI	AIRPFROM	3	☒	☐	☐	☐	☐	Dep. airport
SPFLI	COUNTRYTO	3	☒	☐	☐	☐	☐	Country
SPFLI	CITYTO	20	☒	☐	☐	☐	☐	Arrival city
SPFLI	AIRPTO	3	☒	☐	☐	☐	☐	Dest. airport
SPFLI	FLTIME	6	☒	☐	☐	☐	☐	Flight time
SPFLI	DEPTIME	8	☒	☐	☐	☐	☐	Departure
SPFLI	ARRTIME	8	☒	☐	☐	☐	☐	Arrival Time
SPFLI	DISTANCE	11	☒	☐	☐	☐	☐	Distance
SPFLI	DISTID	3	☒	☐	☐	☐	☐	Distance in
SPFLI	FLTYPE	1	☒	☐	☐	☐	☐	Charter
SPFLI	PERIOD	3	☒	☐	☐	☐	☐	in day(s) later

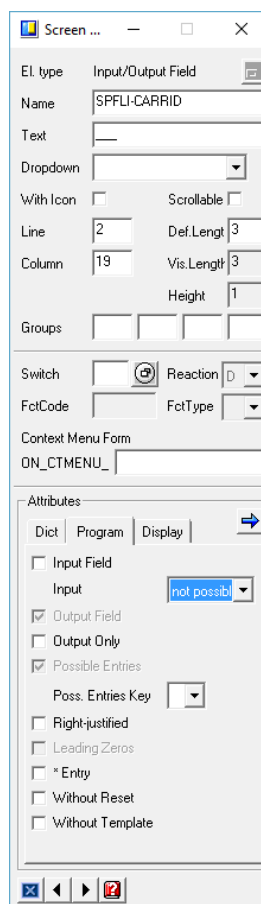
As you already know from the creation of your first screen your mouse cursor has changed now and you can just drop the new input fields onto your new screen. The SAP system comes up with a pop-up, asking you what to do with the **SPFLI-FLTYPE** table field. The question arises as this table field is a **Char 1** field and indicates if the connection is either a charter flight or a regular flight. Please select checkbox, which means every charter flight will be indicated by an activate checkbox later on.

**SPFLI-
FLTYPE**

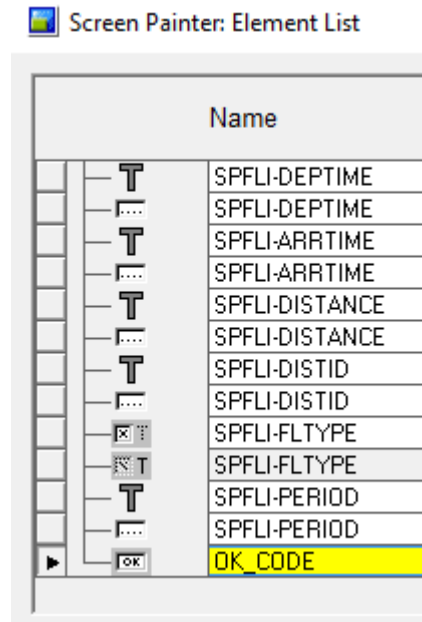


So far you see several input fields in your new screen. Please note that the values in the client input field cannot be changed. Besides the client information we want to ensure nobody makes any changes to the airline and flight number input fields either too. This can be done very easily by changing the attributes of these input fields. Please double click on the airline input field (not the text field!) and change the attribute so that input is not possible:

**Input not
possible**

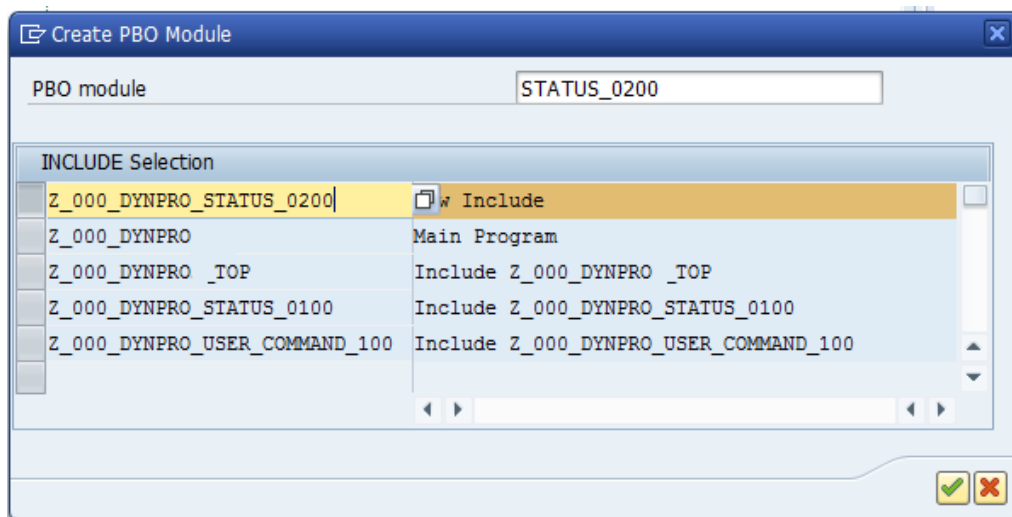


Do the same for the input field of the flight number. The last step is about handling the **ok_code**. Please assign the OK_CODE element to the variable **ok_code** using the **Element List** icon. You may have to scroll down to see the variable.



Save and activate your new screen. Go back to the flow logic of screen 200 and uncomment the line **MODULE STATUS_0200**. Double click on **STATUS_0200** to create the new status bar. Do not forget to change the include name.

PF-STATUS



Uncomment the lines '**PF-STATUS**' and '**TITLEBAR**' in your module and replace the placeholder with '**200**'. Then double click on the first '**200**' to create the status bar. Maintain the short text.

Title bar

```
MODULE status_0200 OUTPUT.
  SET PF-STATUS '200'.
  SET TITLEBAR '200'.
ENDMODULE.
```

Please add the following codes to the buttons:

User Interface	Z_000_DYNPRO	Active
Menu Bar	 	Status for screen 200
Application Toolbar	 	Status for screen 200
Function Keys	 	Status for screen 200
Standard Toolbar	  SAVE  BACK  LEAVE  	

You will create the case instruction for handling the parsed code later on. At first you have to create the title bar. So please save your status bar and go back to your module by clicking Back. Double click on the **200** after title bar and create a new one.

 Display Title

Program	Z_000_DYNPRO
Title Code	200
Title	Connection details



All title

Before you may continue with the case instruction, add the following instruction to your module source code:

```
MODULE status_0200 OUTPUT.
  SET PF-STATUS '200'.
  SET TITLEBAR '200'.

  spfli = wa_flight.
ENDMODULE.
```

*Spfli =
wa_flight*

This ensures that the input fields (named **spfli**) on your screen 200 contain the data from the work area **wa_flight**. The work area was filled in the PAI section of screen 100 and contains the connection details. This data passing was enabled by the Tables statement in the top include file.

Now save, check and activate your new screen and go back to the flow logic tab of your screen 200.

As you already did for the first screen, you have to implement the flow logic for the second screen too. In the **Flow Logic** tab uncomment **USER_COMMAND_0200** and double click on it. Do not forget to change the name of your include by adding a **Z_\$\$_###_** as the first letters.

*USER_COMM
AND_0200*

 Create PAI Module

PAI module	USER_COMMAND_0200
------------	-------------------

INCLUDE Selection

Z_000_DYNPRO_USER_COMMAND_0200	New Include
Z_000_DYNPRO_APPLICATION	Master Program
Z_000_DYNPRO_STATUS_0100	Include Z_000_DYNPRO_STATUS_0100
Z_000_DYNPRO_STATUS_0200	Include Z_000_DYNPRO_STATUS_0200
Z_000_DYNPRO_TOP	Include Z_000_DYNPRO_TOP
Z_000_DYNPRO_USER_COMMAND_0100	Include Z_000_DYNPRO_USER_COMMAND_0100

After you have saved the include add the following source code via Eclipse. This will contain a new case instruction, which handles the parsed code from your status bar.

*Rename
include*

```

MODULE user_command_0200 INPUT.

CASE ok_code.
  WHEN 'LEAVE'.
    LEAVE PROGRAM.
  WHEN 'BACK'.
  WHEN 'SAVE'.
    IF spfli <> wa_flight.
      MODIFY spfli FROM spfli.
    ENDIF.
  ENDCASE.

  CLEAR ok_code.

ENDMODULE.

```

The source code does the following: in case of a 'LEAVE', the program should be ended immediately. In case of a 'SAVE' changed data from the structure 'SPFLI' should be written to the database table 'SPFLI'.

Save, check and activate your new program and test it in a new GUI window.

Choose a connection

Please select an airline and a flight number:

Airline	AA
Flight Number	17

Connection details

Client	200
Airline	AA
Flight Number	17
Country	US
Depart.city	NEW YORK
Dep. airport	JFK
Country	US
Arrival city	SAN FRANCISCO
Dest. airport	SFO
Flight time	6:01
Departure	11:00:00
Arrival Time	14:01:00
Distance	2,572
Distance in	MI
☐ Charter	
n day(s) later	

Remark: As you probably noticed, most of the functionalities to implement a Dynpro program with Eclipse still work through the integrated SAP GUI. The reason for this is that Dynpro programs represent the traditional SAP GUI programs and therefore are still mainly used through it. Hence, Eclipse will not offer functionalities to completely implement Dynpro programs in the native Eclipse IDE. However, it offers useful functions such as the possibility to code the programming logic of the user commands in Eclipse. For large programs, this is a huge advantage, since the code completion and better syntax highlighting improve the coding process.

Task 5: Create an input help for your Dynpro

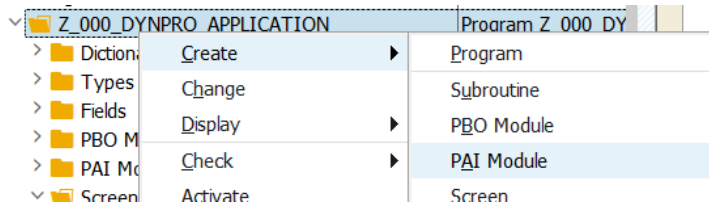
Short description: Create an input help for the flight number field to improve the usability of your program

As you have maybe noticed, it is hard to insert the correct flight number in screen 0100. We will improve that by implementing our own entry help.

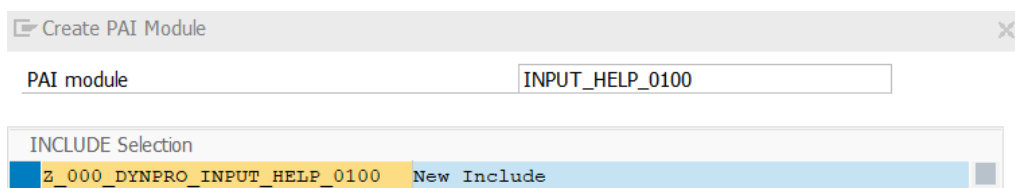
Insert the following lines in the flow logic of your Dynpro screen 0100, after the PAI section and its module:

```
PROCESS ON VALUE-REQUEST.
FIELD spfli-carriid MODULE INPUT_HELP_0100.
```

As before you can create the new module by double-clicking on it (SAP GUI feature). In case you are editing the file via the eclipse IDE you can also open the integrated SAP GUI, right-click on your program name, select create and choose the PAI Module.



Again change the name of the new include file according to the following screenshot.



Open the new ABAP module in the source code editor and change the code according to the following picture:

```
MODULE input_help_0100 INPUT.

DATA progname LIKE sy-repid.
DATA dynnum   LIKE sy-dynnr.
progname = sy-repid.
dynnum   = sy-dynnr.

TYPES: BEGIN OF values,
        carriid TYPE spfli-carriid,
        connid  TYPE spfli-connid,
        countryfr TYPE spfli-countryfr,
        cityfrom TYPE spfli-cityfrom,
        airpfrom TYPE spfli-airpfrom,
        countryto TYPE spfli-countryto,
        cityto   TYPE spfli-cityto,
        airpto   TYPE spfli-airpto,
        deptime  TYPE spfli-deptime,
      END OF values.
DATA values_tab TYPE TABLE OF values.

ENDMODULE.
```

In this snippet we first declare two local variables (**progname** and **dynnum**) that contain the program name and Dynpro number of our application. We could simply state the values of both variables as "Z_\$\$_###_DYNPRO" and the "0100", but this would be bad code design and the module could not be reused in another context. Better to read this information dynamically from the global system fields **sy-repid** and **sy-dynnr**. Afterwards, we define the variable **values_tab** which will store the content of the displayed entry help. The field **connid** is obligatory since the connection id is the final value that will be inserted, the other fields are optional to provide the user with more information when choosing the connection.

To complete the implementation add the following lines between your **values_tab** definition and the **ENDMODULE** statement:

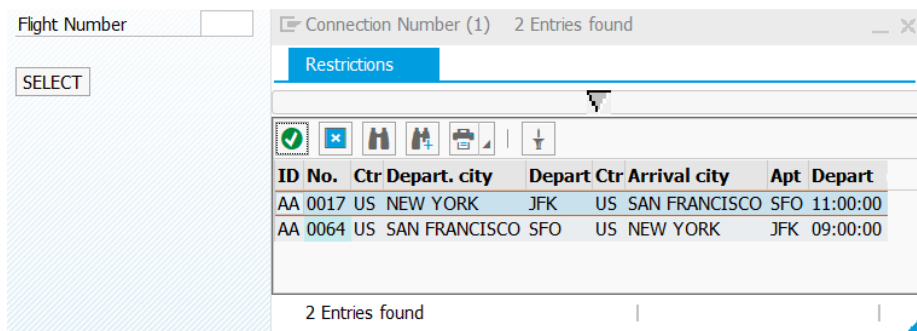
```

SELECT *
FROM spfli
INTO CORRESPONDING FIELDS OF TABLE values_tab
WHERE carrid = 'AA'.

CALL FUNCTION 'F4IF_INT_TABLE_VALUE_REQUEST'
EXPORTING
  retfield      = 'CONNID'
  dynpprog      = progname
  dynpnr        = dynnum
  dynprofield   = 'SPFLI-CONNID'
  value_org     = 'S'
TABLES
  value_tab     = values_tab.

```

In this snippet we query the database for a number of flight connections and store the results in the variable `values_tab`. Afterwards we use a so-called **function module** to display the information of `values_tab` as a pop up window in our dynpro screen. You will learn more on function modules in the "Modularization chapter". The parameter **retfield** specifies which column of the value table will be returned after the user has made his choice. The **dynprofield** specifies the field to which the value will be assigned to. Save, activate and test your application by selecting the field "Flight Number" and clicking on the entry help symbol  or typing **F4**. The result should look as follows:



As you may have noticed the entry help always displays the same two connections. This is because we hardcoded the carrier 'AA' in our select statement. A good input help would dynamically filter the proposed values based on the users pre-selection for the **carrier field**. Go back to the implementation of your input help and add the following lines between the **values_tab** assignment and the SQL statement.

```

DATA: dynpro_values TYPE TABLE OF dynpread,
      field_value LIKE LINE OF dynpro_values.

CLEAR: field_value, dynpro_values.
field_value-fieldname = 'SPFLI-CARRID'.
APPEND field_value TO dynpro_values.

CALL FUNCTION 'DYNP_VALUES_READ'
EXPORTING
  dname      = progname
  dynumb     = dynnum
  translate_to_upper = 'X'
TABLES
  dynpfields = dynpro_values.

READ TABLE dynpro_values INDEX 1 INTO field_value.

SELECT *
FROM spfli

```

As you now the data transport between Dynpro and ABAP layer only happens during PBO and PAI. Therefore we cannot just refer to the screen field spfli-carrid to its current value. This is why we need another function module.

This snippet firstly defines two local variables to store the value of dynpro fields. The clear statement is best practice to ensure that no values are pre-assigned to the new variables. Since we are interested on the value of the airline field (carrid) we define fieldname accordingly. The function module will use this information to read from the correct dynpro field. Afterwards we read from the returned table to store the value of carrid in a flat structure. Finally replace 'AA' in the SQL statement with **field_value-fieldvalue**.

```
SELECT *
  FROM spfli
 INTO CORRESPONDING FIELDS OF TABLE values_tab
 WHERE carrid = field_value-fieldvalue.
```

The complete module should look as follows:

```
MODULE input_help_0100 INPUT.
  DATA progname LIKE sy-repid.
  DATA dynnum LIKE sy-dynnr.
  progname = sy-repid.
  dynnum = sy-dynnr.
  TYPES: BEGIN OF values,
          carrid TYPE spfli-carrid,
          connid TYPE spfli-connid,
          countryfr TYPE spfli-countryfr,
          cityfrom TYPE spfli-cityfrom,
          airpfrom TYPE spfli-airpfrom,
          countryto TYPE spfli-countryto,
          cityto TYPE spfli-cityto,
          airpto TYPE spfli-airpto,
          deptime TYPE spfli-deptime,
        END OF values.
  DATA values_tab TYPE TABLE OF values.
  DATA: dynpro_values TYPE TABLE OF dynpread,
        field_value LIKE LINE OF dynpro_values.

  CLEAR: field_value, dynpro_values.
  field_value-fieldname = 'SPFLI-CARRID'.
  APPEND field_value TO dynpro_values.

  CALL FUNCTION 'DYNP_VALUES_READ'
    EXPORTING
      dyname = progname
      dynumb = dynnum
      translate_to_upper = 'X'
    TABLES
      dynpfields = dynpro_values.

  READ TABLE dynpro_values INDEX 1 INTO field_value.
  SELECT *
    FROM spfli
   INTO CORRESPONDING FIELDS OF TABLE values_tab
  WHERE carrid = field_value-fieldvalue.

  CALL FUNCTION 'F4IF_INT_TABLE_VALUE_REQUEST'
    EXPORTING
      retfield = 'CONNID'
      dynpprog = progname
      dynpnr = dynnum
      dynprofield = 'SPFLI-CONNID'
      value_org = 'S'
    TABLES
      value_tab = values_tab.
ENDMODULE.
```

Challenge: Create options to display (read only) and delete flight data

Task 1: Currently, after an airline and a flight number is chosen on screen 100, clicking the button leads you to screen 200 which allows to modify flight data. The first task of this challenge is to create a second button on screen 100, which leads you to a screen 300. This screen 300 also shall show connection details, but not allow to change flight data. Therefore, input should not be possible in all fields. Additionally, rename the first button text to **Change**, and the second to **Display**.

Task 2: Finally, create a third button on screen 100. This button shall contain the text **Delete**. If this button is clicked, the flight you have chosen in the input fields shall be deleted from the database. For this, you have to assign a function code to the button, and handle the function code in the PAI module.

Your solution screens should look like this:

The first screenshot shows the 'Choose a connection' screen. It has a title bar with 'System' and 'Help' menus. Below the title bar is a toolbar with various icons. The main content area has a title 'Choose a connection' and a prompt 'Please select an airline and flight number:'. There are two input fields: 'Airline' with the value 'AA' and 'Flight Number' with the value '0017'. Below these fields are three buttons: 'Change', 'Display', and 'Delete'.

The second and third screenshots show the 'Connection details' screen. It has a title bar with 'System' and 'Help' menus. Below the title bar is a toolbar with various icons. The main content area has a title 'Connection details' and a list of input fields for flight details. The fields are: Client (100), Airline (AA), Flight Number (17), Country (US), Depart.city (NEW YORK), Dep. airport (JFK), Country (US), Arrival city (SAN FRANCISCO), Dest. airport (SFO), Flight time (6:01), Departure (11:00:00), Arrival Time (14:01:00), Distance (2.572), Distance in (MI), Charter (checkbox), and n day(s) later (0).